

RBDC Ex 3

Project Brief: Ticket Management System for Incidences

Overview

The goal of this project is to build a **Ticket Management System** that enables users to submit, track, and resolve incident tickets. This system will uniquely feature **client identification without requiring login**—achieved through token-based identification, unique links, or email recognition. The system aims to streamline customer support and allow quick and easy issue reporting.

The project will involve full-stack development, including **Next.js** for the frontend, **Django** for the backend, and **PostgreSQL** for data storage. You will also implement **Dockerization** to containerize the services for easy deployment.

Project Expectations

1. Tool Usage

1. Docker for Environment Setup & Deployment

- Use **Docker** to containerize both frontend and backend services, along with the PostgreSQL database.
- Develop `Dockerfiles` for each service and use `docker-compose` to manage the multi-container application.
- **Expectation:** Ensure the app runs smoothly across environments by using Docker and document the setup in a README.

2. Cursor

- Use **Cursor** for efficient code navigation and refactoring across both frontend and backend components.
- **Expectation:** Maintain a well-organized and modular codebase, making use of Cursor for quick navigation between API endpoints, frontend components, and shared modules.

3. ChatGPT 4.0

- Use **ChatGPT** to brainstorm solutions for client identification, design user-friendly incident reporting flows, and troubleshoot bugs during development.
 - **Expectation:** Engage with ChatGPT to explore non-login identification techniques and ensure smooth user experiences.
-

2. Development & Deployment Practices

1. Git for Version Control

- Set up a Git repository with clear branching (e.g., `feature/client-identification`, `feature/ticket-creation`).
- **Expectation:** Regular commits with meaningful messages and feature-based branching to ensure clean collaboration.

2. Postman for API Testing

- Use **Postman** to test all API endpoints for creating, updating, and viewing incident tickets.
- **Expectation:** Ensure all endpoints are tested thoroughly, with correct handling of edge cases and errors, before frontend integration.

3. Digital Ocean for Deployment

- Deploy the system on **Digital Ocean** with containers for frontend, backend, and PostgreSQL services.
 - **Expectation:** Ensure the system is accessible online and document deployment steps clearly.
-

3. Tech Stack and Implementation

1. Next.js 14 (Frontend)

- Build the user-facing interface for ticket creation and status tracking.
- **Pages to Implement:**
 - **Incident Reporting:** A page where users can submit a new ticket.

- **Ticket Status Page:** A page that displays the status and history of submitted tickets.
- **Admin Dashboard:** A restricted section for admins to manage tickets and respond to users.

Expectation: Create a clean, responsive UI that simplifies the ticket reporting process and allows users to view updates easily.

2. Django 5.1 (Backend)

- Set up the backend to handle ticket submissions, updates, and status tracking.
- **Core Models:**
 - **Client:** Represents unique users identified by email or token.
 - **Ticket:** Stores incident details, current status, priority, and history.
 - **Response:** Tracks admin responses and comments on tickets.

Expectation: Build RESTful API endpoints using Django REST Framework to handle ticket operations efficiently.

3. PostgreSQL (Database)

- Use **PostgreSQL** to store tickets, clients, and ticket responses.
- **Expectation:** Design a schema that ensures data integrity and supports queries for ticket history and client lookups.

4. Key Features

1. Client Identification Without Login

- Use **email identification** or **unique ticket links** to identify clients.
- When a ticket is submitted, the system sends a unique link to the user's email. Users can access their ticket status directly through the link without logging in.

User Story:

"As a customer, I want to submit an incident ticket and receive a unique link to track its status without creating an account, so I can report issues quickly."

2. Ticket Submission & Management

- Users can report incidents by providing their name, email, a brief description, and optional attachments.
- Admins can assign priorities (e.g., Low, Medium, High) and track the ticket status (e.g., Open, In Progress, Resolved).

User Story:

"As a product manager, I want to track the status and priority of each ticket, so I can ensure important issues are resolved promptly."

3. Ticket Status Tracking

- Users can access their tickets through a unique link and view their status and any updates from admins.
- Admins can leave comments, update the ticket status, and notify the user automatically.

User Story:

"As a customer service representative, I want to update ticket statuses and leave comments, so users stay informed about the progress."

4. Analytics Dashboard (Optional)

- An admin dashboard provides insights into ticket statistics: average resolution time, open vs. closed tickets, and most common issues.
- Use **charts** to visualize key metrics, such as ticket status distribution and resolution times.

User Story:

"As an admin, I want to view insights about ticket volumes and resolution times, so I can identify bottlenecks and improve our process."

5. Testing & Quality Assurance

1. Test-Driven Development (TDD)

- Write unit tests for models (e.g., Ticket, Client), views, and serializers in Django.
- Use Jest for frontend component testing and end-to-end tests.

Expectation: Achieve at least 70% test coverage, ensuring critical features are well-tested.

2. Error Handling & Feedback

- Implement client-side validation for ticket submission forms (e.g., ensure emails are valid).
- Display clear error messages if ticket submission fails or the ticket link is invalid.

Expectation: Ensure smooth error handling on both frontend and backend, with user-friendly messages.

3. Logging & Monitoring

- Use Django logging to track key events, such as ticket creation and updates.
- Monitor ticket volumes and errors in the deployed system using Digital Ocean's monitoring tools.

Expectation: Implement basic logging to track incidents and errors efficiently.

6. Concepts to Learn & Apply

1. Client Identification Without Login

- Learn how to use **unique tokens** or email-based links for identification.
- **Example:** On ticket submission, generate a unique token that is sent via email. Users use the link to access their ticket without login.

2. Incident Management & Prioritization

- Understand **ticket lifecycle management** (open, in progress, resolved) and **priority handling**.
- **Example:** Use priority levels to ensure critical issues are addressed first.

3. Email Integration & Notification System

- Explore how to send automated emails to users with ticket links and status updates.
- **Example:** Use Django's built-in email functionality to notify users whenever their ticket status changes.

4. Analytics & Insights

- Use simple **data visualization techniques** to generate insights from the ticket database.
- **Example:** Create a dashboard that visualizes open vs. closed tickets and tracks the average time to resolution.

Deliverables & Submission

By the end of this project, you should have:

- A **GitHub repository** containing the complete codebase.
- **Dockerized containers** for frontend, backend, and database.
- A **live deployment** on Digital Ocean, with the system accessible online.
- **Documentation** detailing setup, deployment, and usage instructions.

Submit a link to the **GitHub repository** and the **live app**. Include any known issues, future improvements, and potential extensions in the README.

Good luck, and happy coding! 🎉📧🔧